

# Advanced Natural Language Processing

# LLMs as Agents

Department of Advanced Computer Sciences (DACS)  
Master of AI / DSDM  
Prof dr ir Jan C. Scholtes and Dr. Aki Harma— Course year 2026-2027



Maastricht University

# Learning Goals Today

- Know about Agents in the 1990s
- Understand the definition of an agent
- Understand how to use LLMs as components of agents
- Understand Multi-agent models
- Understand how to address the challenges when using LLMs as agent components.

Agents, have these not been  
around for quit some time  
now?



Maastricht University

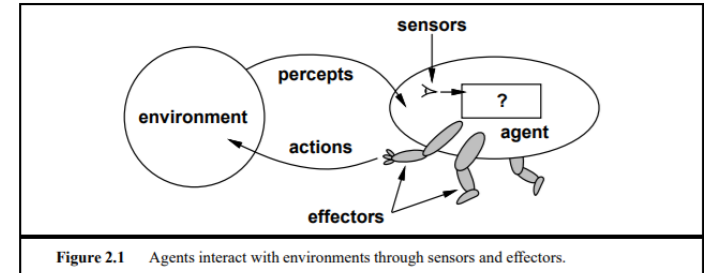
# Agents

AI has been interested in Agents for quite some time. Already in 1995's *Artificial Intelligence: A Modern Approach* (AIMA) by Russell and Norvig, an '**agent**' is defined as:

- **Anything** that
  - (i) **senses** its environment
  - (ii) **perceives** inputs from that environment (e.g., input)
  - (iii) **responds** to those inputs (e.g., reasoning)
  - (iv) **acts** upon the environment through its actions (e.g., output)

Agents can fulfill **goals**, **respond** to percepts or **maximize** their own "happiness"

Agents can be described in terms of **architecture** and in terms of computer **programs**.

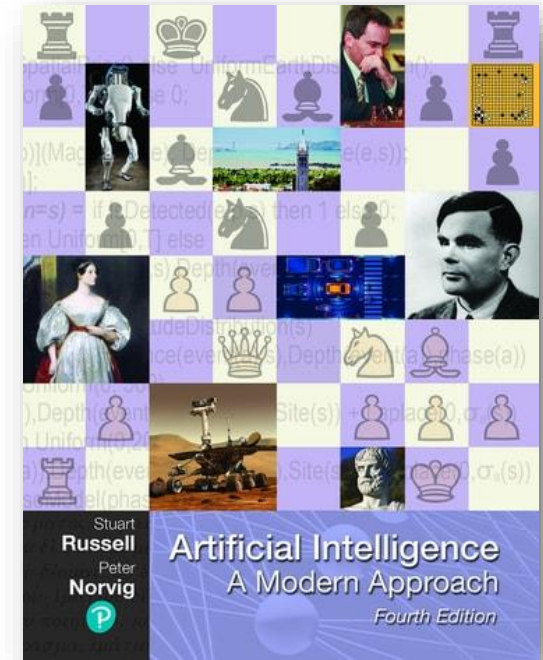


"Of course, all computer programs do something, but computer agents are expected to do more: operate **autonomously**, perceive their environment, persist over a prolonged time period, adapt to change, and create and pursue goals. A rational agent is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome." pp. 23.

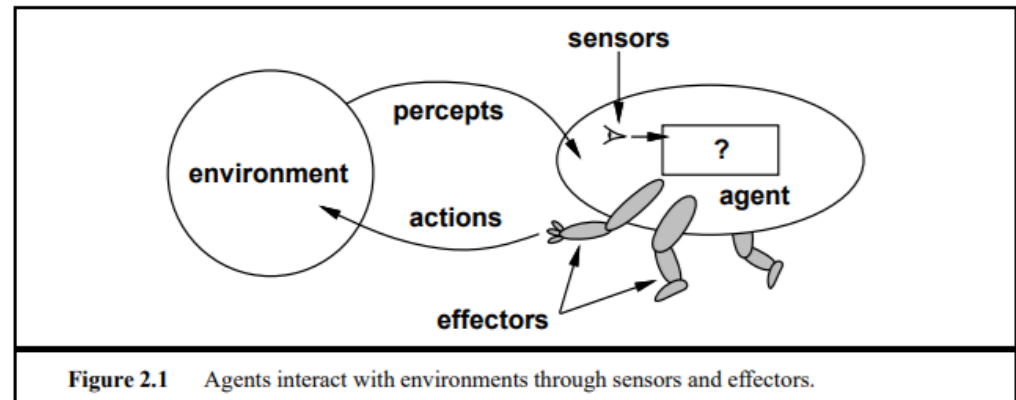
# What an Agent Architecture Requires

Required in an Agent Architecture:

- Memory (short + long term)
- Knowledge access
- Tool use / action execution.
- Planning & control loop
- Goal management
- Verification & guardrails
- User/system interface



Agents are not limited to intelligent or human-like systems—they range from simple reactive devices to complex, reasoning-capable programs. What unites them is that they are autonomous entities operating based on input-output loops within an environment.



# So, what were these Agents?

- In the late 1990s, the word **agent** was everywhere: intelligent agents, software agents, personal assistants, multi-agent systems.
- They were built on databases, symbolic AI for reasoning, expert systems (rules with probabilities), and early distributed computing.
- API's were primitive, Web Access was rare, there were few standards. Lacked robustness in open, noisy environments. Memory was static, knowledge brittle, little learning. Poor language understanding — mostly keyword spotting and rules. No speech recognition. Promises (personal assistants, digital butlers) were far ahead of reality.

1990s agents were clever software wrappers with symbolic reasoning, not adaptive intelligence. They were brittle, narrow, and hard to scale. But they did establish many of the categories (memory, goal, planning, interface) we're still using — only now we have the horsepower to make them succeed.

# The Step to LLM Agents

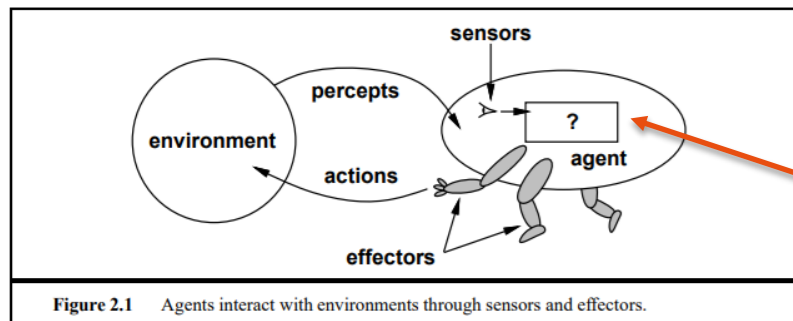
Think of a stand-alone LLM as a brain in a jar. An agent gives it a body (tools), senses (retrieval & APIs), memory (context + DB), and goals (planner). That's when it stops being just a text generator and becomes something that can act.

To go from a bare LLM to an Agent we need to wrap the LLM with:

- Memory (short + long term): conversation history, external storage (vector data bases), past interactions, and RAG.
- Knowledge access (again RAG, APIs).
- Tool use / action execution: call API's to execute actions.
- Planning & control loop: reasoning (the model's own chain of thought, Lectures 11 and 13), planners (split tasks into sub tasks and schedule these) & control loops.
- Goal management: ques and priorities, objective setting, self-reflection module to check progress (can be other LLM agents).
- Verification & guardrails: fact checking modules, safety filters, external (LLMs) critiques.
- User/system interface: conversational (chat, voice, ...).

# From 1990 to 2025 ...

- **1990s agents:** Symbolic, brittle, effective only in **narrow, controlled domains**. Concepts were sound, but tech was immature.
- **2025s agents:** Powered by LLMs + retrieval + APIs. Much more **adaptive, natural, and scalable**, though still challenged by **hallucination, reasoning limits, and reliability**.



Could  
this be  
an LLM?

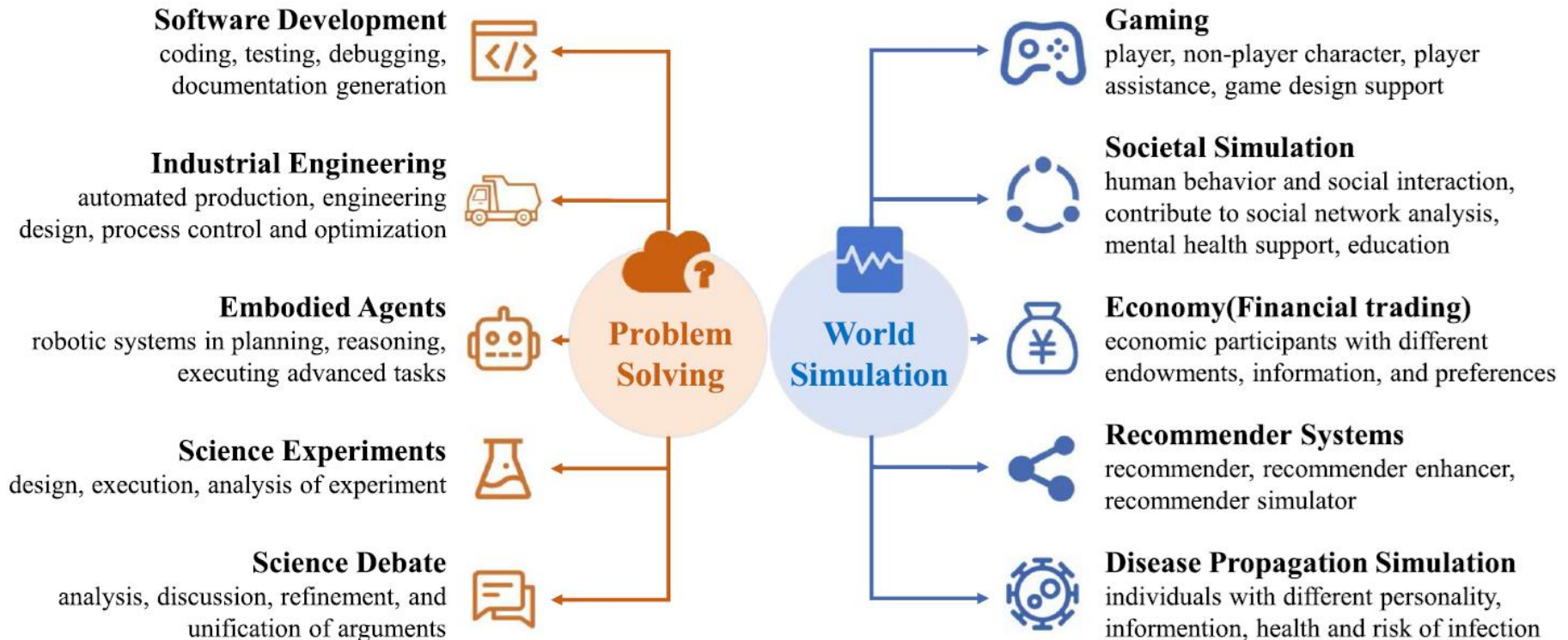
# 1990 Agents versus 2025 Agents: details

| Component                                 | 1990s Agents (Russell & Norvig era)                                                                                                | 2020s LLM-Agents                                                                                                                                                       |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Memory (short &amp; long-term)</b>     | Short-term: rule engines, working memory (Soar, CLIPS). Long-term: handcrafted KBs, ontologies, Prolog databases. Static, brittle. | Short-term: conversation context in prompt. Long-term: vector DBs + Retrieval-Augmented Generation (RAG), external logs. Dynamic, scalable.                            |
| <b>Knowledge Access (retrieval, APIs)</b> | Early APIs (CORBA, DCOM, KQML). Data access mainly fixed DBs or curated knowledge bases. Primitive web scraping.                   | Rich APIs, tool calling, web access. Plug-in ecosystems (LangChain, OpenAI functions). Real-time retrieval from massive corpora.                                       |
| <b>Tool Use / Action Execution</b>        | Scripts, wrappers, messaging protocols (FIPA). Agents could trigger software actions but limited adaptability.                     | Native tool calling from LLMs, Python execution, API orchestration. Can compose multi-step tool workflows.                                                             |
| <b>Planning &amp; Control Loop</b>        | Symbolic planners (STRIPS, PDDL). BDI models (Belief-Desire-Intention). Often computationally heavy → fallback to reactive agents. | Prompt-based reasoning: Chain-of-Thought, Tree-of-Thought, skeleton reasoning. Agents with explicit control loops (ReAct, AutoGPT). More scalable but still imperfect. |
| <b>Goal Management</b>                    | Explicit goal stacks or rule-based priorities. Simple desires like “book meeting.” Conflicts hard to resolve.                      | Natural language goals parsed by LLM. Dynamic decomposition into subtasks. Multi-objective agents possible, with arbitration strategies.                               |
| <b>Verification &amp; Guardrails</b>      | Deterministic rules & exception handling. Correctness = programmer’s responsibility. No hallucinations (but brittle failures).     | Fact-checking modules, RAG grounding, multi-agent debate, self-consistency checks. Still hallucination risk but more systematic guardrails.                            |
| <b>User / System Interface</b>            | GUIs, email filters, embodied agents (Clippy, MIT Media Lab avatars). Dialogue based on finite-state or frame systems. Very rigid. | Conversational AI with LLMs. Natural, open-domain dialogue. Multimodal interfaces (text, speech, vision). Smooth integration with chat UIs, apps, voice.               |

## So, Agents are:

- Agentic AI using LLMs, don't just generate text, but can perceive, decide, act, and learn in an autonomous loop.
- Agentic AI is about turning LLMs into **dynamic problem-solvers** that interact with tools, remember context, plan actions, and iteratively improve.

# Applications of (Multi-Agent) Systems



**Fig. 7** Diverse application of the llm-based multi-agent system

# Agents & LLMs

Let's take a closer look

# LLMs as Agents

LLMs become agents when placed in a loop with (long-term) **memory, goals, reasoning, actions and feedback loops**.

They can then perceive instructions from its environment, plan steps, interact with tools, and adapt to feedback — much like traditional agents in AI, but with far broader knowledge and linguistic flexibility.

# From LLMs to Agents

- So, from providing an answer to a question to executing a task by generating computer code (SQL) or using an API.
- Formally, in reinforcement learning or agent-based AI:
  - **Environment:** the external world (APIs, tools, user inputs).
  - **Policy:** the decision function mapping states → actions.
  - **Reward:** signals indicating success.
- An LLM can serve as the **policy**, using text as its interface to reasoning and action.



# A Formal Definition of Agents ...

Let's formalise the idea of **LLMs as agents** using reinforcement learning (RL) and sequential decision-making notation. This will provide a **mathematical intuition** behind why plugging an LLM into an agent loop is not just engineering, but grounded in established AI theory.

In reinforcement learning, an **agent** interacts with an **environment** that can be formalised as a Markov Decision Process (MDP):

- $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$
- $\mathcal{S}$ : set of states.
- $\mathcal{A}$ : set of actions.
- $P(s' | s, a)$ : transition probability.
- $R(s, a)$ : reward function.
- $\gamma \in [0,1]$ : discount factor.

The agent follows a **policy**  $\pi(a | s)$  that maps states to actions. The goal is to maximise expected return:

$$J(\pi) = \mathbb{E}_{\pi} \left[ \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

# LLMs as Policies

For an LLM-agent, we reinterpret:

- **State**  $s_t$ : the **prompt** or context window at time  $t$ , including user instructions, environment observations, and agent memory.
- **Action**  $a_t$ : the **output text** generated by the LLM — which may be:
  - a natural-language utterance,
  - a structured command (e.g., SEARCH("population of Maastricht")),
  - or a plan step ("Step 1: find data...").
- **Policy**  $\pi_\theta(a \mid s)$ : the probability distribution over next tokens or strings, parameterised by LLM weights  $\theta$ .

So the LLM is the **stochastic policy**:  $\pi_\theta(a_t \mid s_t) = (Wh_t)$

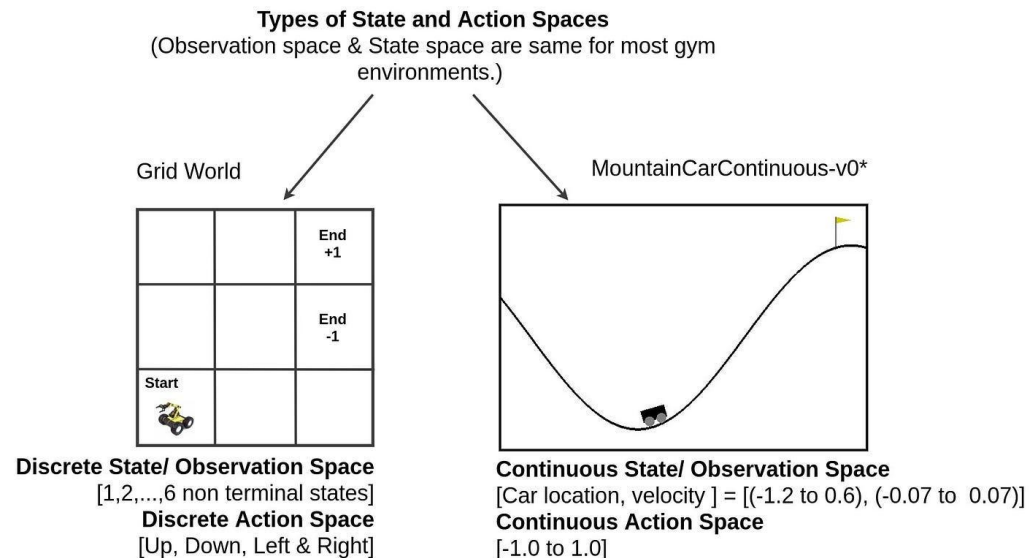
- where  $h_t$  is the hidden state encoding of the prompt at time  $t$ .



# From discrete state spaces to continuous

Traditional agents move between discrete symbolic states.

How about LLM agents?



\* MountainCar-v0 is the discrete action space version of this.

# Do LLMs have states?

LLMs *do* have states, but they are **not discrete symbolic states** like in classical RL. Instead, the state is:

- the **entire prompt**,
- internally encoded as a **continuous vector in transformer hidden space**,
- updated on each token via **deterministic continuous dynamics**.

This means LLM-based agents fit naturally into **continuous-state, continuous-action reinforcement learning**, also known as **Partially Observable Markov Decision Processes (POMDPs)** with **continuous observation and action spaces**.

# LLM Agents as Partially Observable Markov Decision Process (POMDP)

LLM-based agents can be formalized as POMDPs:

- True **state**  $s_t$ : unknown environment state
- **Observation**  $o_t$ : context window / prompt
- **Policy**  $\pi_\theta(a_t \mid o_t)$ : LLM generating next token/tool call
- **Action**  $a_t$ : natural-language text or structured tool invocation
- **Transition**  $P(s_{t+1} \mid s_t, a_t)$ : environment + tool effects
- **Reward**  $R(s_t, a_t)$ : evaluator, feedback, or success signal

# LLMs as Policies

In LLM agents, both states and actions are continuous:

- **Observation/state:** high-dimensional hidden vector  $h_t$
- $h_t$  encodes entire **prompt**; updated continuously per token
- **Action** space  $A \approx V^*$ : all possible token sequences
- **Policy**  $\pi_{\theta}(a_t \mid o_t) = \text{Softmax}(W h_t)$

Implies continuous-state, continuous-action RL

We will look at some of these  
later in the lecture



**There is more theoretical grounding from:**

Deep RL for continuous control (e.g. Deep Deterministic Policy Gradient (DDPG))

POMDPs with belief states

Sequence decision-making / decision transformers

Policy-as-language-model (Ouyang et al., 2022; Wen et al. 2024)

# Why LLMs as Agents?

- **Natural reasoning interface:** text is expressive enough to encode plans, decisions, and feedback.
- **General knowledge:** LLMs bring world knowledge without handcrafted rules.
- **Few-shot adaptability:** can adapt behaviour via prompting or demonstrations.
- **Scalability:** once an LLM-agent loop is defined, it can operate across domains (software engineering, scientific discovery, robotics).

# Why add Agentic Architecture to stand-alone LLMs?

- **Hallucinations (Misinformation):** LLMs sometimes produce *confident-sounding but false information*.
- **Knowledge Cut-off:** Models only know about data up to their training cutoff date. They lack awareness of any events or facts after that.
- **Limited Context Window:** There's a limit to how much text you can feed into the model at once (maybe a few thousand tokens). This means the model can't *truly remember* unlimited conversation history or large documents unless you manage that externally. Long conversations or large documents risk the model forgetting earlier details.
- **No Internet or Tools Access:** By default, an LLM in isolation can't perform external actions – it can't browse the web, use a calculator, or call an API. It's constrained to its training knowledge and textual output.
- **One-Shot Reasoning:** In a single prompt-response, the model might not do complex reasoning reliably. It often answers in one go, which can lead to errors on multi-step problems (math, logical puzzles, code execution)
- **No Self-critique or Memory of Mistakes:** If the model makes an error, it won't know it by itself in a single-turn interaction. There's no innate mechanism for it to reflect on or correct its answer after producing it.

Some of these problems are off-course addressed by RAG, Internet Lookup and CoT, as we discussed before. These components are also part of an Agentic architecture. But Agents go further...

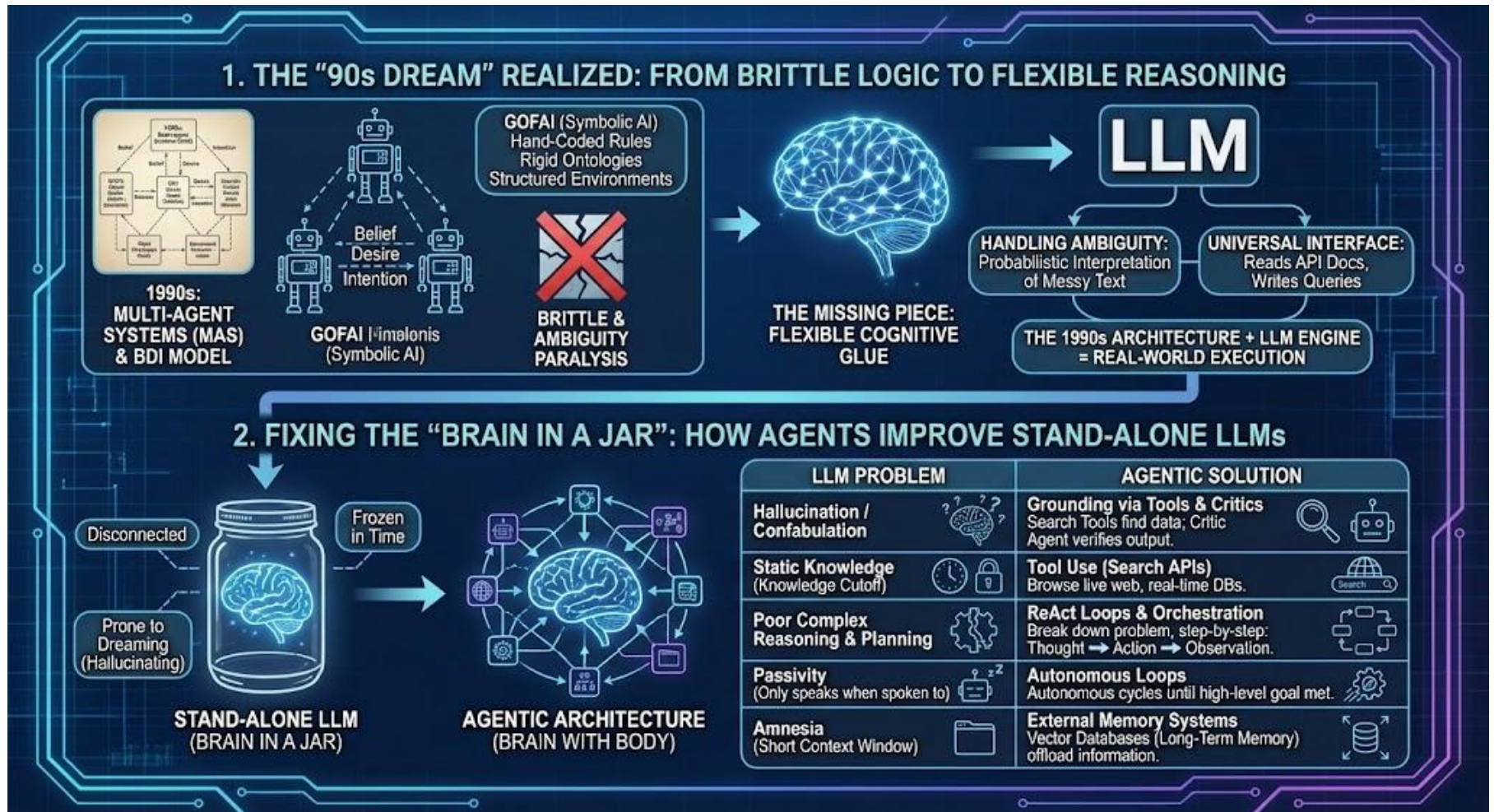
Agentic Architectural components can assist reducing these problems



# Symbiosis or Mutual Benefit

- So, we can say LLMs can make many of the original goals of Agentic systems from the 1990s come true.
- Agentic architectures are beneficial to reduce many of the problems stand-alone LLMs have.

# Symbiosis or Mutual Benefit



So, we can say LLMs can make many of the original goals of Agentic systems from the 1990s come true. Agentic architectures are beneficial to reduce many of the problems stand-alone LLMs have.

# The Components of an LLM-based Agent

Perception  
Memory  
Action Interface  
Goal Management  
Reasoning & Planning  
Learning and Adaptation  
Verification & Guardrails



Maastricht University

# Components of LLM Agents

## 1. Perception (input processing)

- LLMs naturally handle diverse inputs expressed as text: instructions, structured data, dialogue.
- Multimodal models extend this to images, audio, and video.

## 2. Memory

- Short-term memory: the context window.
- Long-term memory: external vector databases (e.g., Pinecone, FAISS), used to retrieve past knowledge.
- Episodic memory: logs of prior interactions.

## 3. Action interface

- Through **tool use**: calling APIs, querying databases, executing code.
- Through **environment control**: issuing commands in a simulator, game, or robotic system.
- This often follows the **ReAct paradigm** (Reason + Act), where the LLM first explains reasoning, then outputs a tool call.



# Components of LLM Agents

## 4. Goal/Task Management

- Task queue and prioritization.
- Objective setting (from user or system).
- Self-reflection modules to check progress.

## 5. Reasoning & Planning

- Reasoning: the model's own chain of thought (Lectures 11 and 13), prompted or trained.
- Planners: Explicit structures to decompose tasks into subgoals.
- Control loop: Decide next step, call tool or LLM, verify, repeat.

## 6. Learning & adaptation

- Fine-tuning, preference tuning (RLHF, DPO) and RL on agent tasks (Lecture 13) refine behaviour.
- Online adaptation is still challenging but research into process-based rewards and self-reflection addresses this.

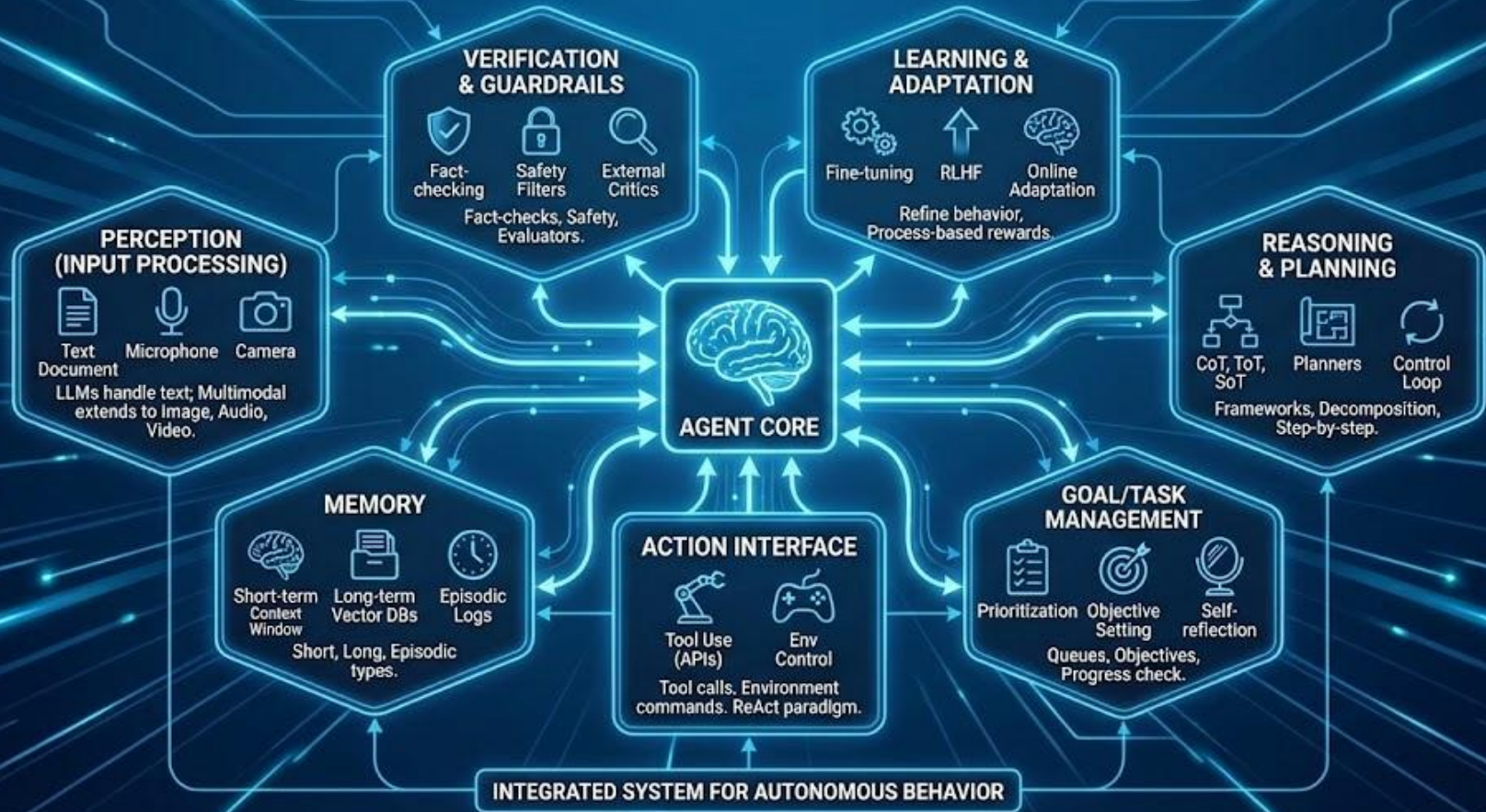
## 7. Verification and Guardrails

- Fact-checking modules.
- Safety filters (ethical, legal constraints).
- External critics or evaluators ("self-consistency checks," debate agents).

Let's discuss these components in more detail in the subsequent slides



# COMPONENTS OF AN AI AGENT



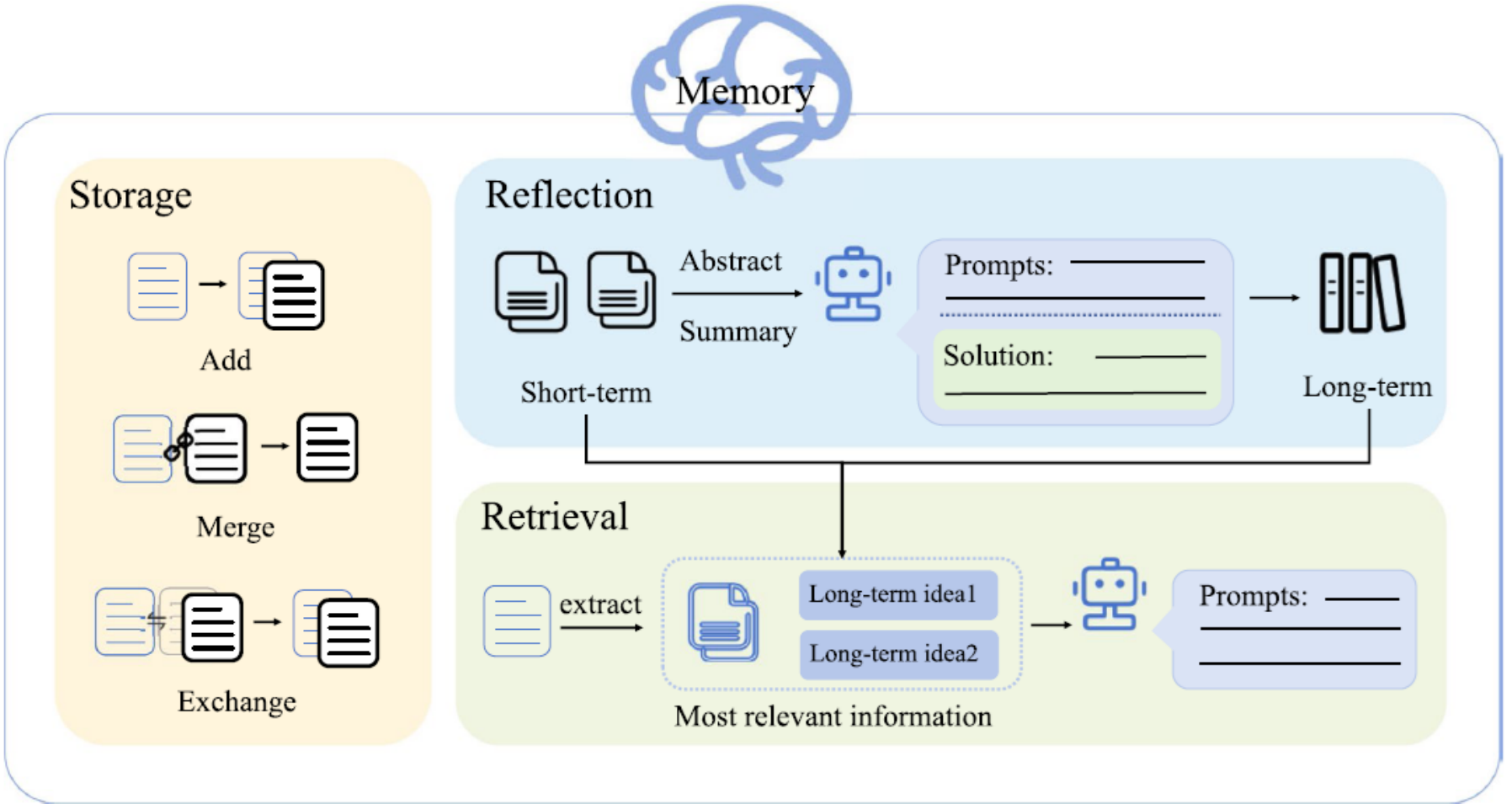
# 1. Perception (input processing)

- With today's LLMs, input processing functionality is out of the box: chatting in NLP, speech, images, document uploads (including Tables in XLS and OCR for images and bitmap PDF's), etc.

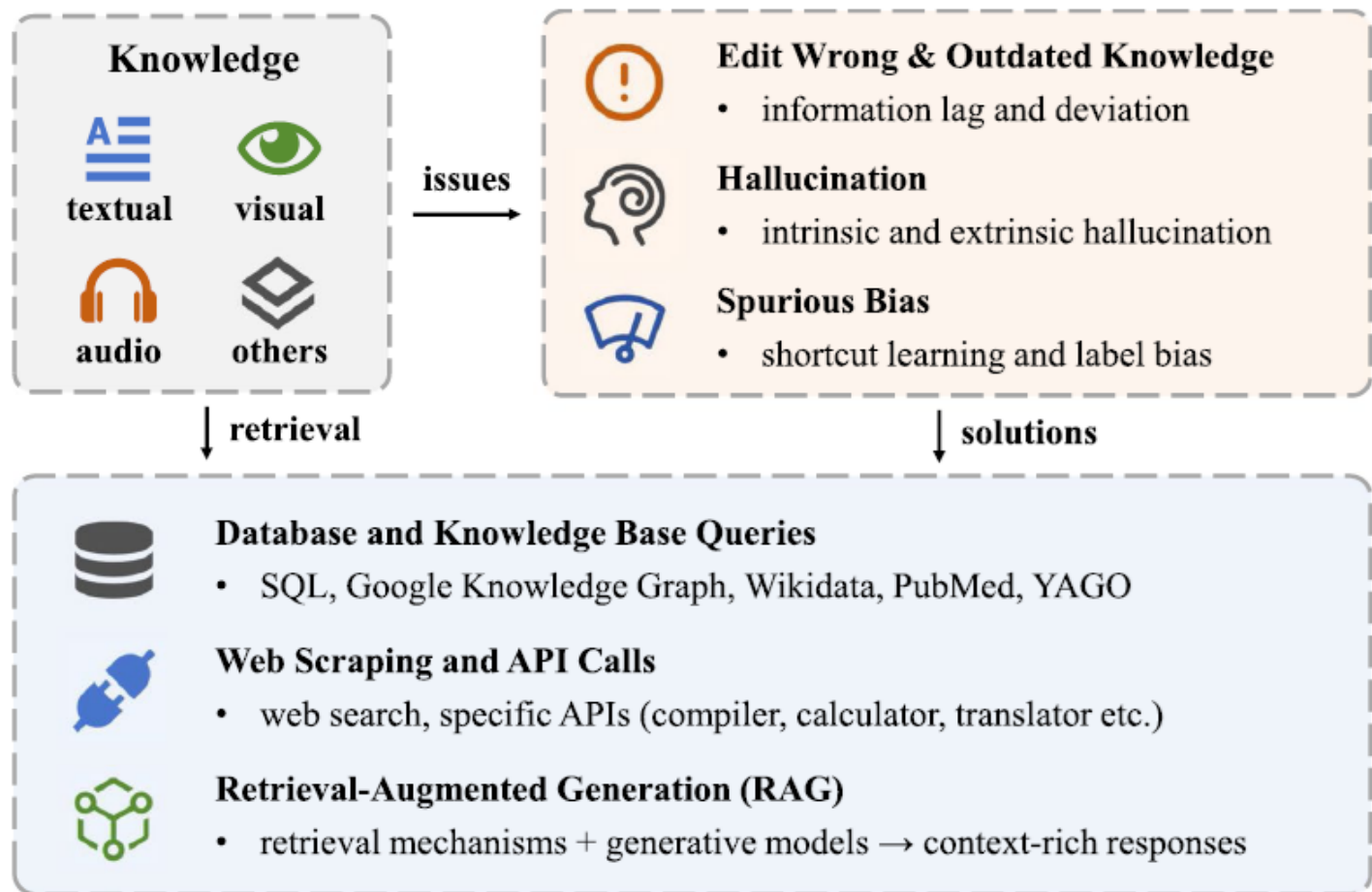
## 2. Memory

Stand-Alone LLMs are state less. Once the prompt ends, they forget everything. So, we need a form of long-term memory:

- Short-term memory: Conversation history, kept in the context window.
- Long-term memory: External storage (vector databases, knowledge bases, logs).
- Episodic memory: Past interactions.
- Semantic memory: Retrieved knowledge via RAG.



**Fig. 2** The operational mechanism of the memory module



**Fig. 3** Knowledge utilization

### 3. Action interface (aka Tools)

The difference between a LLM and an Agent is that an Agent actually *does things*. It does not just talk!

So we need:

- Tool calling (APIs, Python execution, databases).
- Plugins or function calling (as in OpenAI's tool APIs).
- Environment interaction (e.g., robotics controllers, operating system).

with tools, the agent can reach  
beyond its own limitations

## 4. Goal Management: The Classic View (1990s Agents)

Back in the **BDI (Belief–Desire–Intention)** architecture (very influential in Russell & Norvig’s early days):

- **Beliefs** = what the agent *knows* (state of the world, memory).
- **Desires** = what the agent *wants* (possible goals).
- **Intentions** = what the agent is *committed to pursue* (active goals).

Goal management was about **moving from desires → intentions**.

**Techniques used then:**

- **Goal stacks / priority queues:** The agent picked one goal at a time from a list, often with fixed priority rules (e.g., “safety first, then efficiency”).
- **Rule-based arbitration:** If two goals conflicted (“avoid collision” vs. “reach target”), rules dictated which won.
- **Scripts and queries:** Users gave explicit desires (“find cheapest flight”), which agents mapped to an action plan (search database, compare prices).

**Effectiveness:**

- Worked for **narrow goals** where conflict was rare.
- But if the agent had multiple competing desires (e.g., “minimize cost” vs. “minimize travel time”), arbitration became brittle.
- No general reasoning about trade-offs → agents could deadlock or oscillate between goals.

# Why is Goal Management Hard

- **Conflicting goals:** “Be on time” vs. “Drive slowly for safety.”
- **Dynamic environments:** A goal set earlier might no longer make sense as the world changes.
- **Scalability:** Adding new goals often required rewriting arbitration logic.
- **No meta-goals:** Agents lacked the ability to reflect and reprioritize (“which goal is more important right now?”).

# Goal Management with LLMs

LLM-based systems handle goals differently:

- **Natural language goals:** Users express them freely (“Plan my trip under €500”).
- **Goal decomposition:** LLMs can break a high-level goal into **subgoals** (budget, flights, hotels, activities).
- **Dynamic reprioritization:** Some frameworks (AutoGPT-style agents) re-evaluate goals at each loop iteration.
- **Arbitration strategies:** Modern approaches combine:
  - Rule-based filters (hard constraints: budget  $\leq$  €500).
  - Scoring functions (utility maximization: pick the plan with highest satisfaction).
  - Human-in-the-loop when goals are ambiguous.

## Even with LLMs:

- Conflicts remain: Models aren’t inherently good at resolving trade-offs without explicit scoring systems.
- Drift happens: Agents may “forget” goals mid-process if context isn’t reinforced.
- Overgeneralization: If given too vague a goal (“make me successful”), they spin in circles.



# Break



Agent Smith from the *Matrix* - 1999

# 5. Reasoning & Planning in LLMs

- LLMs are “local optimizers”: they generate the next most likely token given the prompt. That produces text that sounds coherent, but doesn’t guarantee a correct multi-step solution.
- Multi-step tasks (like “plan a trip,” “debug code,” or “summarize patents then compare them”) require keeping track of intermediate goals, sub-steps, and dependencies.
- Without an explicit planning scaffold, LLMs can drift, lose track of context, or contradict themselves.

# A generalized notion of 'reasoning'

Unlike humans, LLMs (mostly) only have one mechanism (token generation) for perception, intuitive inferences, and symbolic reasoning; everything is effortful and takes a forward pass



## GPT-4o

In this image, we see a young boy sitting on the floor, looking down at a pile of broken dishes. He appears to be sad or remorseful. In front of him, a woman, likely his mother or caregiver, is kneeling and looking at him with a concerned, questioning expression. Her body language, with her hands open and her facial expression, suggests that she's asking him something or expressing surprise or frustration about the broken dishes.

(<https://language-agent-tutorial.github.io/>)

# A generalized notion of 'reasoning'

Unlike humans, LLMs (mostly) only have one mechanism (token generation) for perception, intuitive inferences, and symbolic reasoning; everything is effortful and takes a forward pass



## GPT-4o

In this image, we see a young boy sitting on the floor, looking down at a pile of broken dishes. He appears to be sad or remorseful. In front of him, a woman, likely his mother or caregiver, is kneeling and looking at him with a concerned, questioning expression. Her body language, with her hands open and her facial expression, suggests that she's asking him something or expressing surprise or frustration about the broken dishes.

Perception

(<https://language-agent-tutorial.github.io/>)

# A generalized notion of 'reasoning'

Unlike humans, LLMs (mostly) only have one mechanism (token generation) for perception, intuitive inferences, and symbolic reasoning; everything is effortful and takes a forward pass



## GPT-4o

In this image, we see a young boy sitting on the floor, looking down at a pile of broken dishes. He appears to be sad or remorseful. In front of him, a woman, likely his mother or caregiver, is kneeling and looking at him with a concerned, questioning expression. Her body language, with her hands open and her facial expression, suggests that she's asking him something or expressing surprise or frustration about the broken dishes.

Perception

Intuitive Inference

(<https://language-agent-tutorial.github.io/>)

# A generalized notion of 'reasoning'

Unlike humans, LLMs (mostly) only have one mechanism (token generation) for perception, intuitive inferences, and symbolic reasoning; everything is effortful and takes a forward pass



## GPT-4o

In this image, we see a young boy sitting on the floor, looking down at a pile of broken dishes. He appears to be sad or remorseful. In front of him, a woman, likely his mother or caregiver, is kneeling and looking at him with a concerned, questioning expression. Her body language, with her hands open and her facial expression, suggests that she's asking him something or expressing surprise or frustration about the broken dishes.

Perception

Intuitive  
Inference

Reasoning

One may alternatively call this 'thought' to avoid the over-loaded term of 'reasoning,' at the risk of further anthropomorphizing machines

12

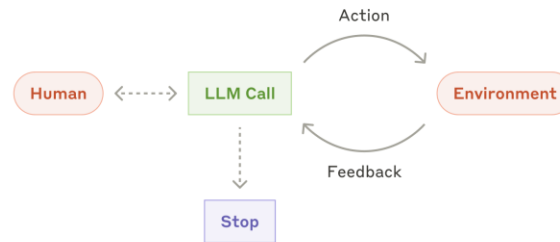
# Where does the reasoning live?

- **Who writes the chain of reasoning?** The prompt (Lecture 11), the training (Lecture 13), or the harness and the environment (this lecture).
- **Who judges it?** Humans, a reward model, a checker, or the environment: a test that passes, a page that shows the answer, a user who objects.
- **When is the compute spent?** In training (RL on agent tasks) or at run time (think longer, retry, search).

**2023:** the harness supplied the reasoning. Chain-of-thought, tree-of-thoughts and ReAct prompts were wrapped around a frozen model.

**2025:** the model supplies the reasoning (RL-trained chains, Lecture 13) and the harness supplies the world: tools, state, memory, verification, permissions, budget.

# Interleaved thinking: reason, act, reason again



- **Inside one response:** the model thinks, calls a tool, reads the result, thinks again, calls the next tool. The chain of thought and the tool calls are one sequence.
- **o3 and o4-mini (April 2025)** were “trained to use tools through reinforcement learning”, including cropping and zooming into images inside the chain of thought. Claude 4 (May 2025) exposes interleaved thinking between tool calls; Kimi K2 Thinking (November 2025) chains 200 to 300 tool calls in one task.
- **The ReAct loop of 2022** (next slides) is the same loop. What changed: the “Thought” is now the model’s native reasoning, and the actions are structured tool calls instead of text the harness has to parse.
- **Consequence for the harness:** it no longer decides when to think; it decides what the model may touch, what it gets to see, and when to stop.

# Another Example

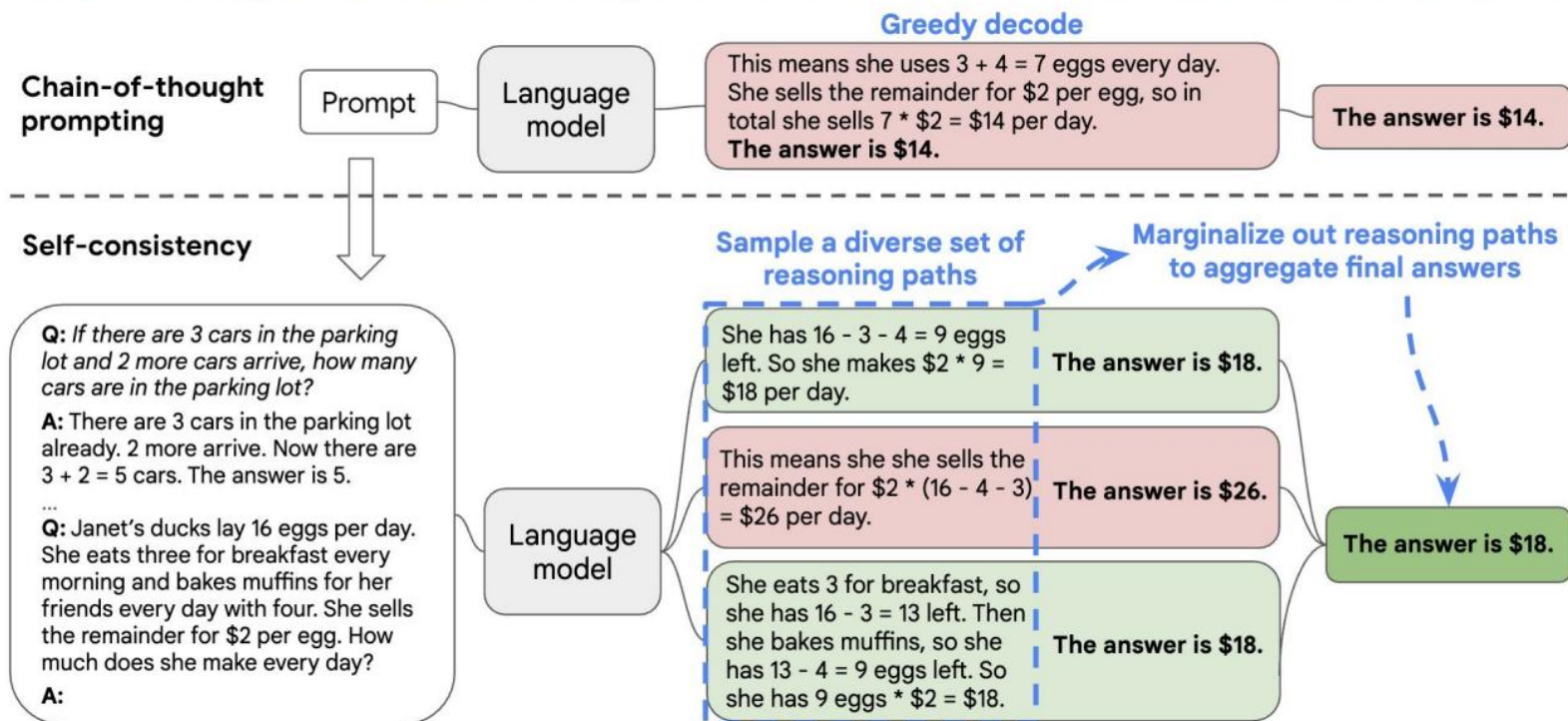
A bare LLM can answer: “What’s  $12 \times 34$ ?” (trivial).

For “Plan my 3-day trip to Kyoto under €500, and make sure hotels are near train stations” →

- CoT: might write a long plan but miss constraints.
- ToT: might try multiple itineraries and pick one.
- Planner + control loop: breaks task into subgoals (budget, transport, hotels), queries APIs, verifies costs, then synthesizes a final plan.

# Adding Self-Consistency to the picture

**Self-Consistency: select the response with the most consistent final answer**



Note: the selection is only based on the final answer, the reasoning paths do not need to be the same across different sampled responses.

# Planning: (simplified) definition

Given a goal  $G$ , decides on a sequence of actions  $(a_0, a_1, \dots, a_n)$  that will lead to a state that passes the goal test  $g(\cdot)$

General trends in planning settings for language agents

- Increasing expressiveness in **goal specification**, e.g., in natural language as opposed to formal language
- Substantially expanded or open-ended **action space**
- Increasing difficulty in automated **goal test**

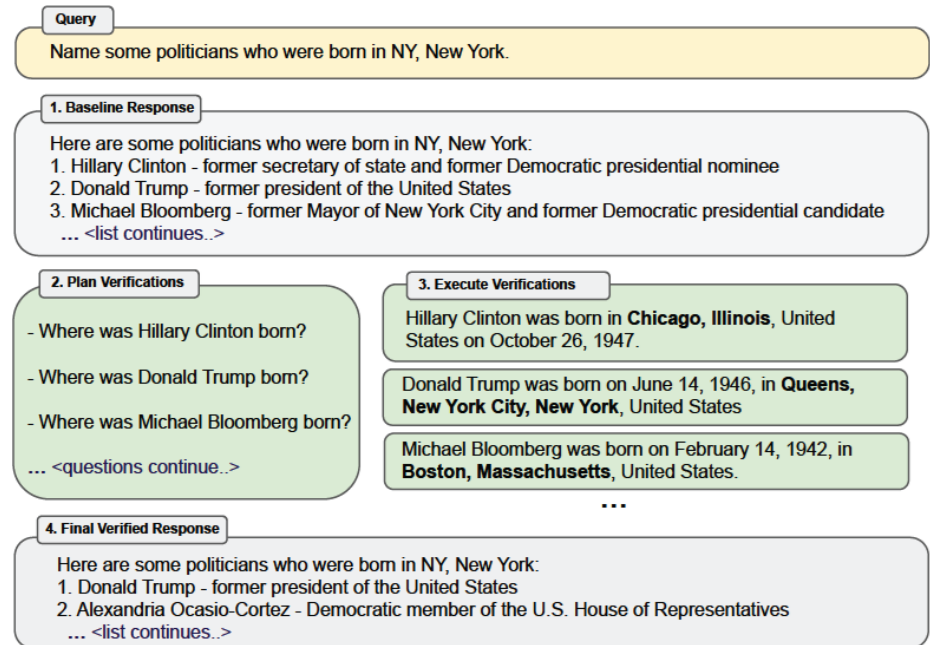
# Reflection and Self-Improvement

- **Reflection** is the pattern where the agent **examines its own output or intermediate work, and then improves it.**
- **How reflection works:** After the model gives an answer or a draft, you prompt it to *critique and refine* that answer. For instance:
  - *Initial attempt:* The agent answers a question or produces some output.
  - *Reflect prompt:* “Critique the above output. Where might it be incorrect or improved?”
  - The model then identifies potential errors or improvements in its answer.
  - *Refinement:* You then prompt, “Given that feedback, please revise your answer.” The model generates a corrected answer.
- This can be done in a loop (hence terms like *iterative refinement* or *self-healing* prompts).

# Chain of Verification (CoVe)

- CoVe refers to a multi-stage process in which a system verifies intermediate reasoning steps, outputs, or decisions before moving forward.
- It is often implemented as a pipeline of checks, either by:
  - the same model, revisiting its own outputs (→ self-reflection),
  - a secondary model, trained or prompted to critique,
  - external tools, such as rule-based validators or retrievers.

Source: Chain-of-Verification Reduces Hallucinations in LLMs. Dhuliawala et al., 2023.



A Chain of Verification might involve:

- Document retrieval: Use RAG to find supplier reports. → Verify: Are these docs from Q2? Are they relevant?
- Information extraction: Find entities and contract clauses. → Verify: Is the contract breach clearly stated?
- Answer synthesis: Generate a natural language response. → Verify: Run fact-checking module or a debate-style critic.
- Output validation: Legal constraints? Risk flags? → Verify: Apply safety filter before surfacing to the user.

# ReAct: Reasoning + Acting

ReAct (Reasoning + Acting) is a prompting strategy that enhances large language models (LLMs) by integrating reasoning and tool use in a single decision-making loop.

LLMs can:

- Reason in natural language (e.g., via Chain of Thought)
- Use tools or APIs (when prompted or integrated in a system)

But standard prompting methods often separate reasoning from actions, or omit one entirely. ReAct combines both into an interleaved structure, allowing the agent to:

- Think through problems step by step
- Act (e.g., search a database, call a calculator)
- Reflect and continue reasoning
- Choose next actions based on what it learned
- This is especially useful in interactive environments or agentic systems.

# How ReAct works

**Question:** What's the capital of the country with the largest population in Africa?

**Thought 1:** I need to find which country in Africa has the largest population.

**Action 1:** Search("most populous country in Africa")

**Observation 1:** Nigeria

**Thought 2:** Nigeria is the most populous country in Africa.

**Action 2:** Search("capital of Nigeria")

**Observation 2:** Abuja

**Answer:** The capital is Abuja.

## Components:

- **Thought:** The LLM reasons out loud.
- **Action:** The LLM takes an action (e.g., call a tool).
- **Observation:** The result of the action is returned.
- This loop continues until the final **Answer**.



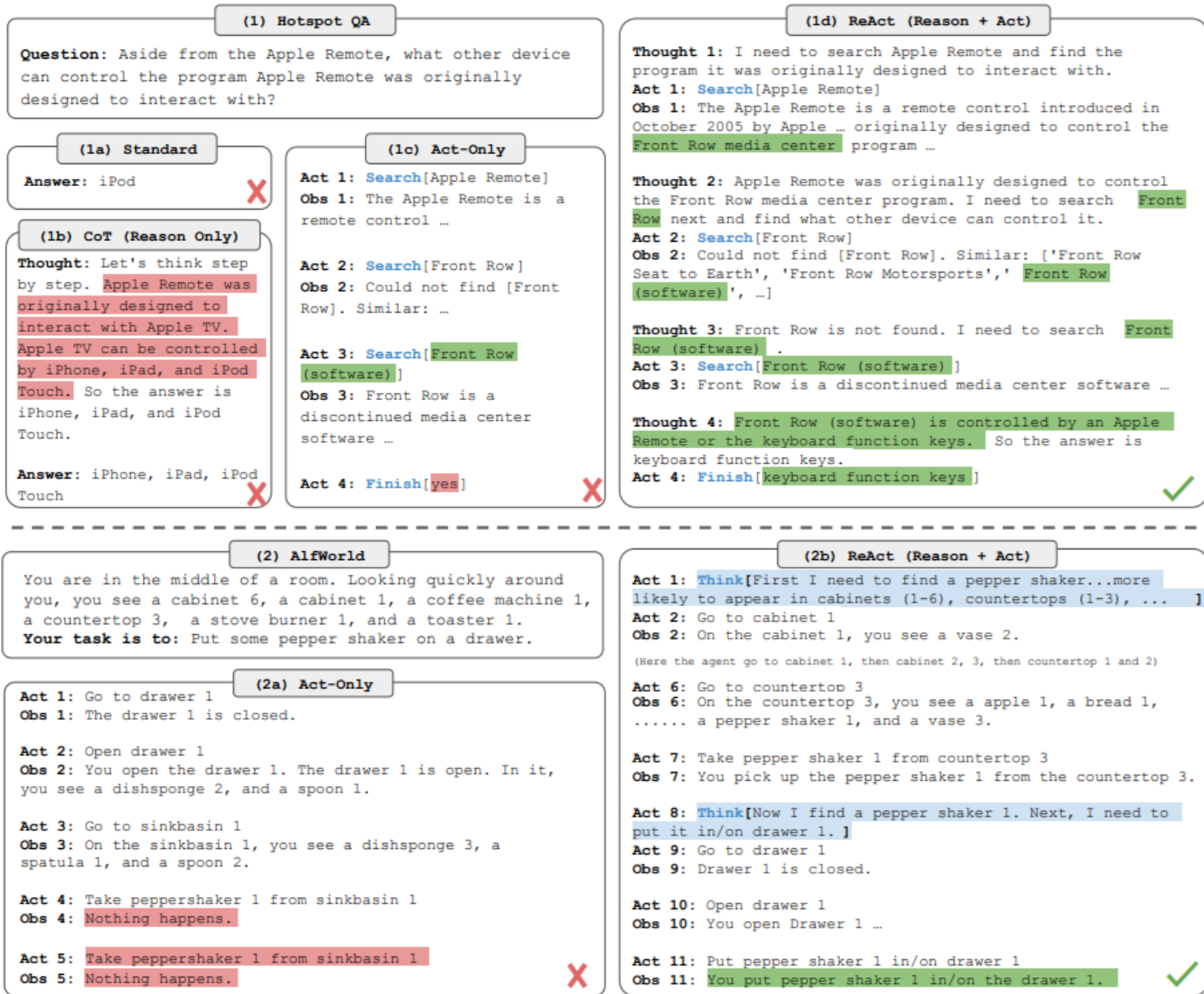
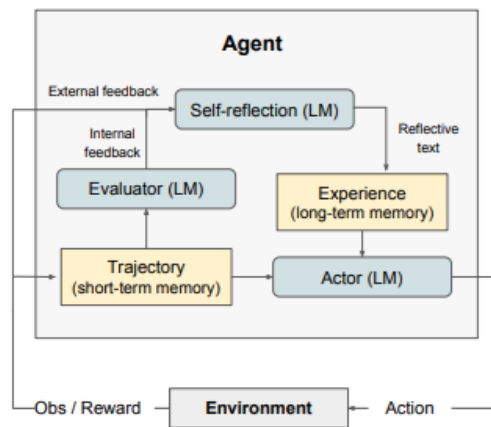


Figure 1: (1) Comparison of 4 prompting methods, (a) Standard, (b) Chain-of-thought (CoT, Reason Only), (c) Act-only, and (d) ReAct (Reason+Act), solving a HotspotQA (Yang et al., 2018) question. (2) Comparison of (a) Act-only and (b) ReAct prompting to solve an AlfWorld (Shridhar et al., 2020b) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

# Reflexion



## Algorithm 1 Reinforcement via self-reflection

```
Initialize Actor, Evaluator, Self-Reflection:  
 $M_a, M_e, M_{sr}$   
Initialize policy  $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$   
Generate initial trajectory using  $\pi_\theta$   
Evaluate  $\tau_0$  using  $M_e$   
Generate initial self-reflection  $sr_0$  using  $M_{sr}$   
Set  $mem \leftarrow [sr_0]$   
Set  $t = 0$   
while  $M_e$  not pass or  $t < \text{max trials}$  do  
  Generate  $\tau_t = [a_0, o_0, \dots, a_i, o_i]$  using  $\pi_\theta$   
  Evaluate  $\tau_t$  using  $M_e$   
  Generate self-reflection  $sr_t$  using  $M_{sr}$   
  Append  $sr_t$  to  $mem$   
  Increment  $t$   
end while  
return
```

Figure 2: (a) Diagram of Reflexion. (b) Reflexion reinforcement algorithm

Reflexion is a framework that allows a large language model to improve its own performance over time by using feedback from past actions and results.

The core idea is that the agent can **analyze failures**, reflect on them in natural language, and **adapt its future behavior**. This reflection process serves as a kind of **meta-cognition**—thinking about thinking.

## Why Reflexion?

LLMs often perform poorly on complex tasks that require multi-step reasoning because:

- They may hallucinate or make logical mistakes.
- They don't learn from previous failures in standard prompting setups.
- They lack internal memory or mechanisms for self-correction.

**Reflexion** addresses this by allowing the model to:

- Look back on what it did wrong.
- Verbalize an explanation or correction (a “reflection”).
- Use that insight to modify how it approaches the task next time.

# Reflexion

Source: Reflexion: Language Agents with Verbal Learning.  
Shinn et al., 2023. <https://arxiv.org/pdf/2303.11366>

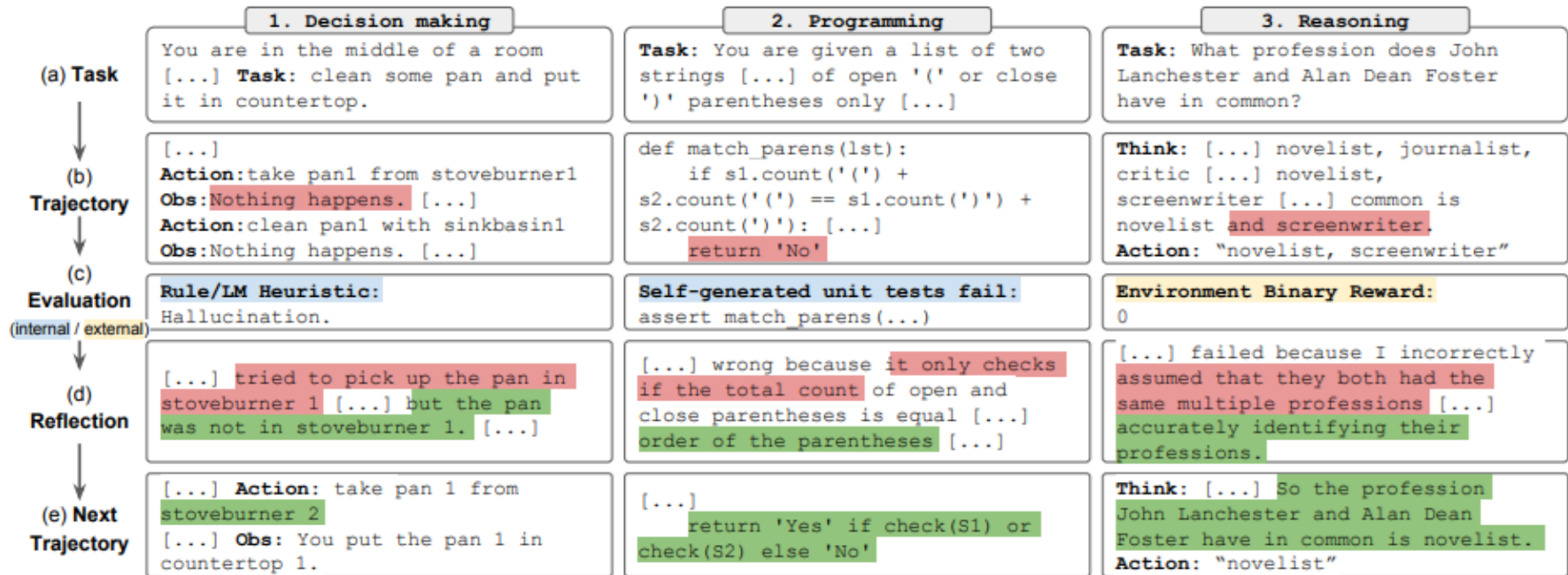


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.

## How Reflexion Works (Simplified Pipeline):

- Agent attempts a task (e.g., solve a puzzle, answer a question).
- Evaluate the result (Was it correct? Did it fail?).
- Reflect: If it failed, generate a verbal reflection — a natural language analysis of what went wrong.
- Adapt future reasoning using this reflection. The next attempts incorporate the reflection to avoid repeating the same mistake.

This can be repeated over multiple rounds. After each round, the agent gets better.

**Why it works on HumanEval (80 to 91 percent):** the agent runs unit tests, so each reflection starts from a real failure.

# AutoGPT

AutoGPT is an (research driven) experimental open-source project that transforms a Large Language Model (LLM), such as GPT-4, into a fully autonomous agent capable of pursuing goals without constant user input by giving an LLM access to tools (web, memory, file system) and letting it decide how to accomplish goals through reasoning, planning, and self-critique.



AutoGPT wraps an LLM in a framework that gives it:

- A goal-setting interface: The user defines high-level goals (e.g., “Research the best electric cars under \$40,000 and write a summary report”).
- Autonomous planning: The agent decomposes this goal into subgoals and plans steps without human prompts.
- Tool use: The agent executes actions (e.g., web search, file saving) through plugins or APIs.
- Memory: It stores context over time to remain coherent across steps.
- Self-reflection: It can evaluate its own progress and revise its strategy.

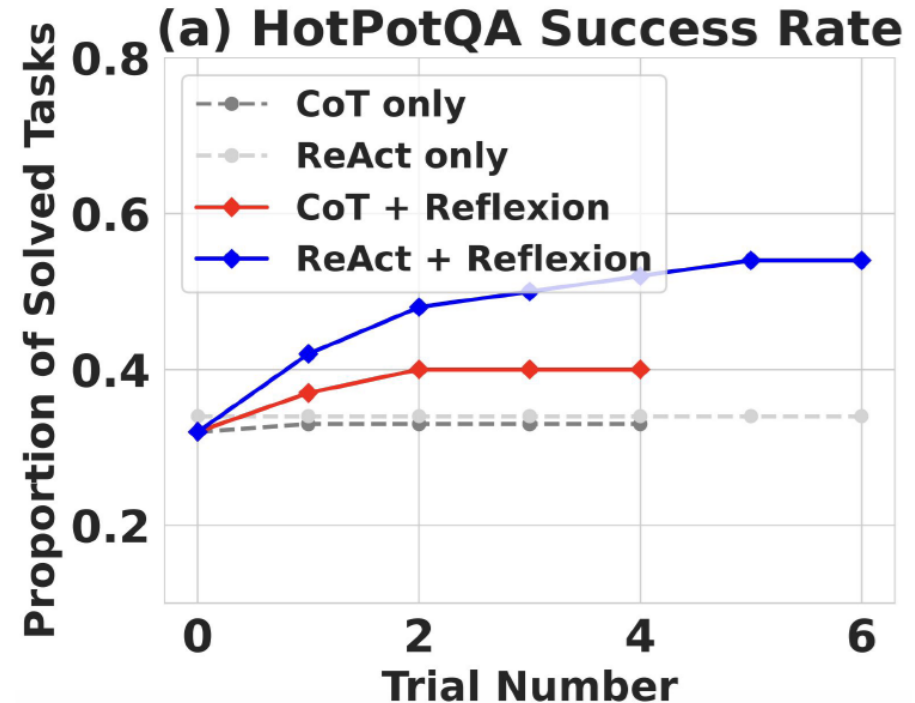
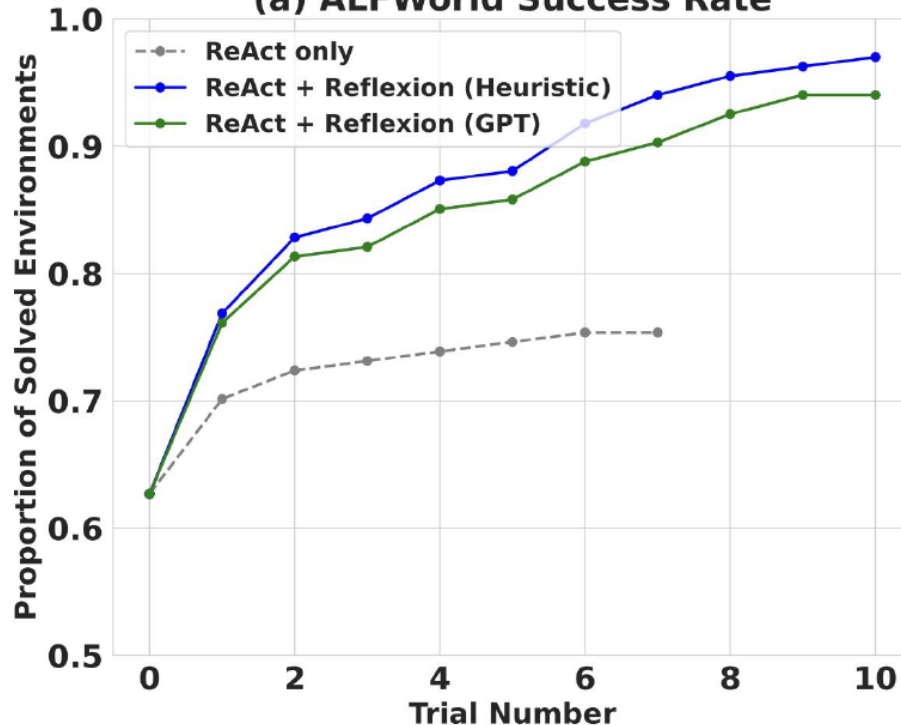
This differs from traditional CoT (Chain of Thought) or ReAct agents, which require task-specific prompting or orchestration.

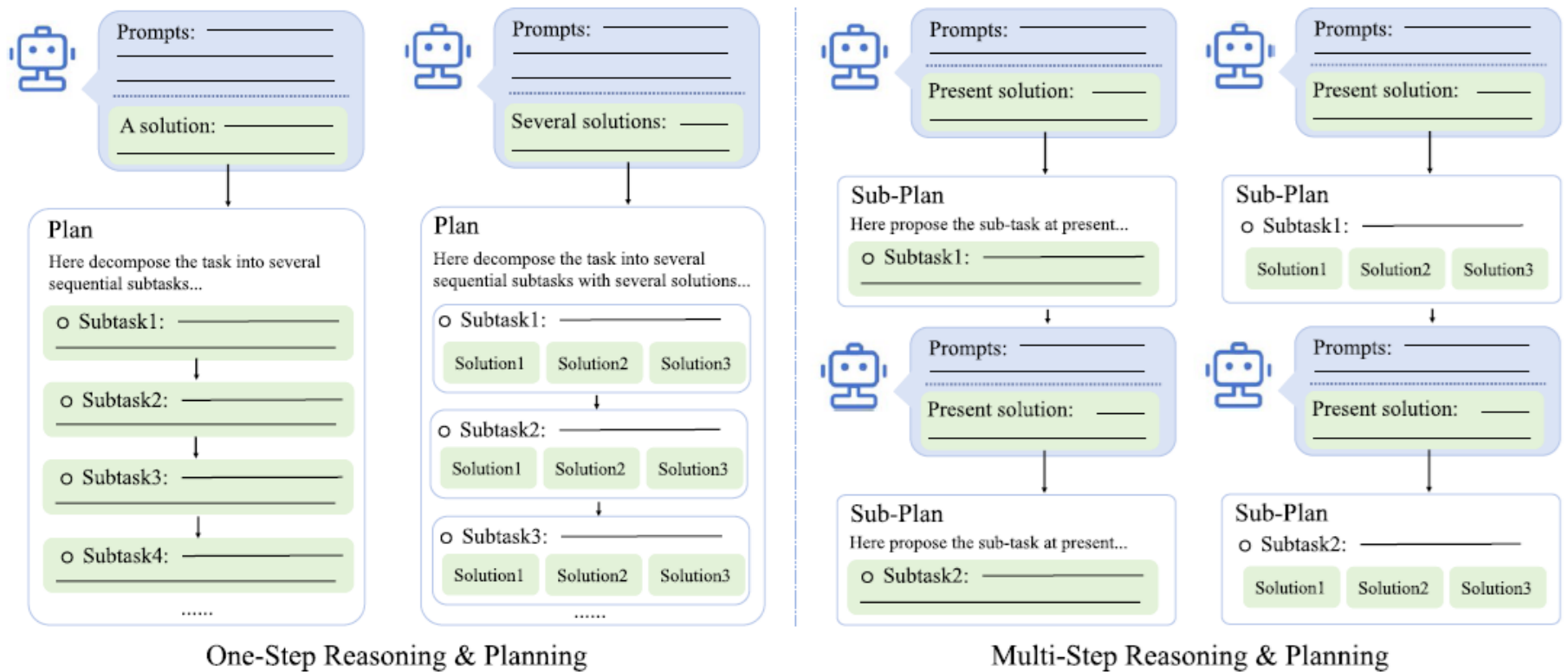
# Comparing CoT, ReAct, Reflexion and AutoGPT

| Feature                        | ReAct | CoT | Reflexion           | AutoGPT |
|--------------------------------|-------|-----|---------------------|---------|
| Combines reasoning & action    | ✓     | ✗   | ✓ (post-hoc)        | ✓       |
| Tool use integrated            | ✓     | ✗   | ✗ (unless combined) | ✓       |
| Immediate observation feedback | ✓     | ✗   | ✗                   | ✓       |
| Explicit control loop          | ✓     | ✗   | ✓                   | ✓       |
| Reflection after failure       | ✗     | ✗   | ✓                   | ✓       |

# Results from these methods

(a) ALFWorld Success Rate

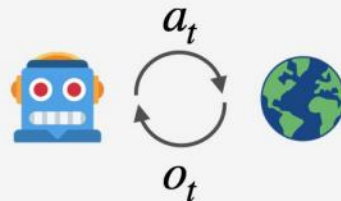




**Fig. 4** Approaches for reasoning and planning with large language models

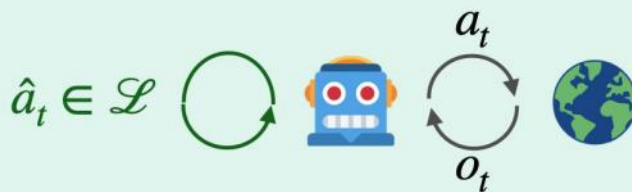
# Why is reasoning special for agents?

**Traditional agents:** action space  $A$  defined by the environment



- **External feedback**  $o_t$
- Agent context  $c_t = (o_1, a_1, o_2, a_2, \dots, o_t)$
- Agent action  $a_t \sim \pi(a | c_t) \in A$

**ReAct:** action space  $\hat{A} = A \cup \mathcal{L}$  augmented by reasoning



- $\hat{a}_t \in \mathcal{L}$  can be any language sequence
- Agent context  $c_{t+1} = (c_t, \hat{a}_t, a_t, o_{t+1})$
- $\hat{a}_t \in \mathcal{L}$  only updates **internal context**

a = action, o = observation

# Reasoning & Planning: Planners

Planners: Explicit structures to decompose tasks into subgoals.

Beyond prompting, you can give the agent an **external planning mechanism**:

- **Symbolic planners** (like STRIPS, PDDL) → classic AI methods for decomposing goals into subgoals.
- **Task decomposition frameworks** → e.g. LangChain's plan-and-execute, BabyAGI-style loops.
- **Hierarchical decomposition**: Break “plan a conference” → into subtasks: venue, speakers, flights, marketing.

Here the LLM isn't just spewing text — it's proposing **subgoals** in a structured format, and the agent executes them one by one.

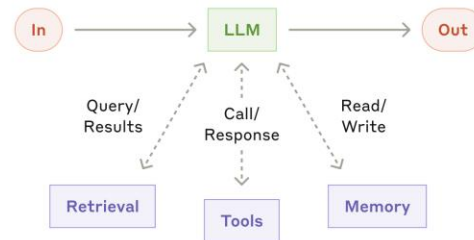
# Reasoning & Planning: Control Loops

**Control loop:** Decide next step, call tool or LLM, verify, repeat.

Instead of one-shot prompting, an agent has a **loop**:

- **Decide next action:** Based on current state (goal, memory, retrieved info).
- **Act:** Call a tool (search engine, calculator, API) or ask the LLM to expand reasoning.
- **Observe:** Get new results, update memory/state.
- **Verify:** Check whether the result is consistent/factual.
- **Repeat** until the goal is reached or the agent halts.
- This loop is what distinguishes an **agent** from a **prompt**. It's a cybernetic cycle of **Plan → Act → Observe → Reflect**.

# Anatomy of a harness in 2026



- **Workflows:** code decides the path: prompt chaining, routing, parallel calls, orchestrator and workers, evaluator and optimiser.
- **Agents:** the model decides the path, in a loop with tools and feedback (Anthropic, December 2024).
- **The harness owns:** the tools and the environment; memory across sessions (notes, compaction, retrieval); permissions and guardrails; verification (tests, linters, judges); the budget (steps, tokens, effort).
- **The model owns:** the next step and which tool to call, and the reasoning in between.
- **Context engineering:** the harness decides what the model gets to see: compaction when the window fills, notes files that survive a reset, sub-agents with a clean context that report back a summary.

# The 100-line agent

- **mini-SWE-agent:** a prompt; the model emits one bash command; the harness runs it and appends the output; repeat. About 100 lines of Python. More than 74 percent on SWE-bench Verified with a frontier model (2025).
- **Harness ablation (2026):** with the same strong model and the same time budget, four elaborate open-source harnesses gave no advantage over a minimal one; giving the model a coding environment was the only intervention that mattered.
- **The bitter lesson for agents:** hand-built reasoning scaffolds helped weak models and stop mattering for strong ones; the labs now train tool use into the model (next slides).
- **What the harness is still for:** the sandbox, the tests, the permissions, the budget. Within one model the spread between harnesses is still large, so the harness decides reliability, not the ceiling.

# Language agent planning: web agents

## Task Description:

Show me the reviews for the auto repair business closest to 10002.

## Action Sequence:

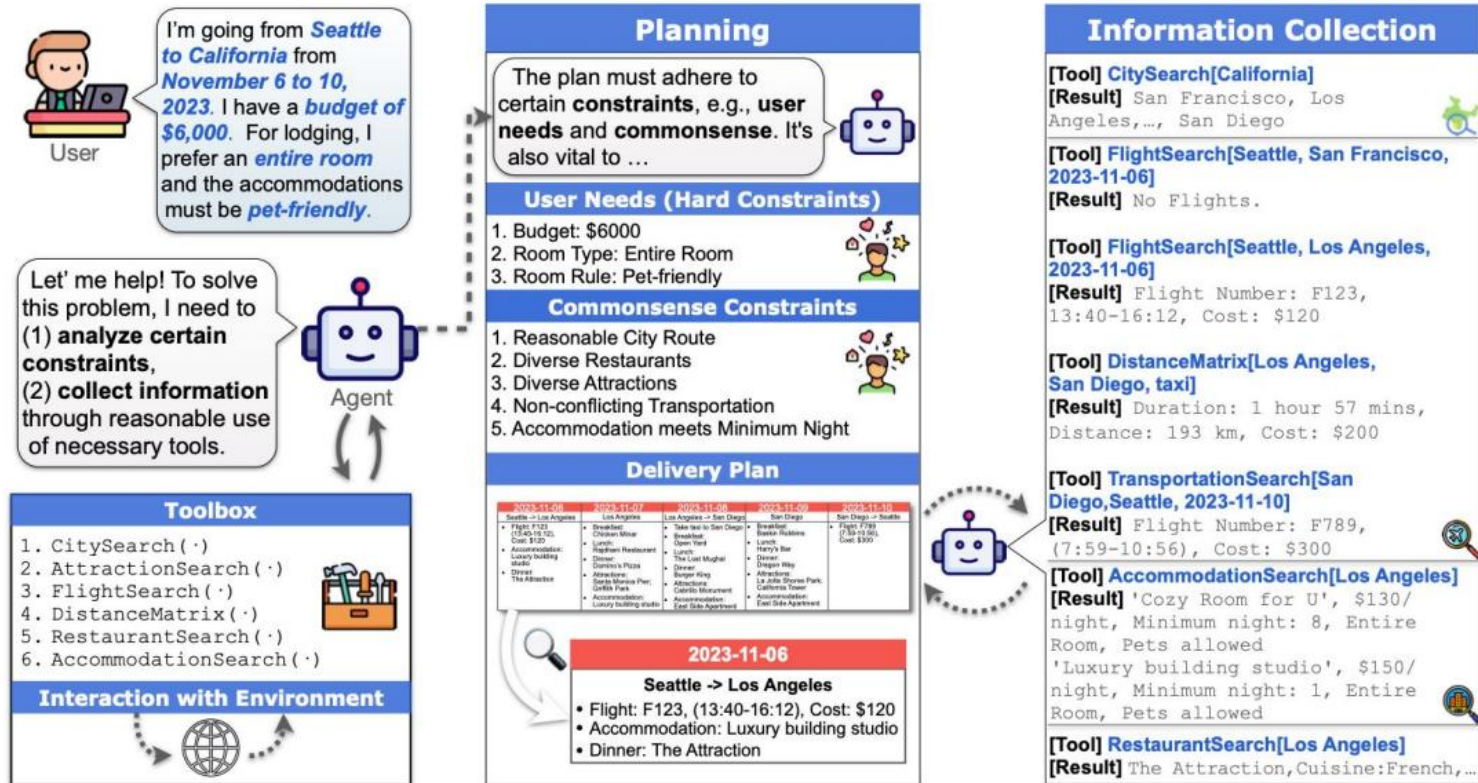
| Target Element                          | Operation            |
|-----------------------------------------|----------------------|
| 1. [searchbox] Find                     | TYPE:<br>auto repair |
| 2. [button] Auto Repair                 | CLICK                |
| 3. [textbox] Near                       | TYPE:<br>10002       |
| 4. [button] 10002                       | CLICK                |
| 5. [button] Search                      | CLICK                |
| 6. [switch] Show BBB Accredited only    | CLICK                |
| 7. [svg]                                | CLICK                |
| 8. [button] Sort By                     | CLICK                |
| 9. [link] Fast Lane 24 Hour Auto Repair | CLICK                |
| 10. [link] Read Reviews                 | CLICK                |

## Webpage Snapshots:



Deng et al., "Mind2Web: Towards a Generalist Agent for the Web." NeurIPS (2024)

# Language agent planning: travel planning



Xie et al., "TravelPlanner: A Benchmark for Real-World Planning with Language Agents." ICML (2024)

# 6. Learning & adaptation

## Fine-tuning and RLHF

- **Fine-tuning:** This involves continuing to train a pretrained language model on a more specific dataset to adapt it to a new domain or behavior. For example, you might fine-tune an LLM to behave more like a legal advisor or technical assistant.
- **Reinforcement Learning with Human Feedback (RLHF):** Here, human preferences guide the model toward more desirable outputs. After pretraining, humans score the outputs of the model, and reinforcement learning (e.g., PPO) is used to reward outputs that align with these preferences.
- **Use in agents:** RLHF or DPO shape behaviour and safety (Lecture 13); since 2025 the models themselves are trained with RL on agent tasks, with the environment as the judge (next slides).

# Rewards

Different reward designs are possible:

- **Outcome-based rewards:** only final success/failure (e.g., solved the task?).
- **Process-based rewards:** intermediate correctness of reasoning steps; used for ranking and search, hard to use as a training signal at scale (Lecture 13).
- **Heuristic shaping:** penalise hallucinations, encourage tool use.

Formally:

$$R(s_t, a_t) = \begin{cases} +1 & \text{if correct final answer} \\ r_{\text{step}} & \text{for valid intermediate reasoning/action} \\ -1 & \text{for invalid or harmful action} \end{cases}$$

# Agentic RL: training the model to use tools

- **RLHF is a one-step bandit:** prompt, answer, reward. An agent episode is a POMDP (slides on the formal definition): many turns, partial observations, one sparse reward at the end.
- **2025, the first wave:** Search-R1 (multi-turn search, retrieved tokens masked from the loss); ReTool (interleaved code execution, AIME 67 percent after 400 RL steps); SWE-RL (Meta, RL on GitHub history, 41 percent on SWE-bench Verified); Kimi K2 (synthetic agentic data from thousands of tools, then RL); Qwen3-Coder (RL on executable tasks in 20,000 parallel environments).
- **The reward is what the environment can verify:** the tests pass, the page shows the answer, the task checker says done; rubric judges where nothing can be checked.
- **Same optimiser family as Lecture 13** (GRPO), new problems: environment throughput, tools that change under the model, reward hacking against weak tests (special-casing the test), long rollouts.

# Credit assignment in long episodes

- **One reward for 200 steps:** trajectory-level GRPO gives every token of every step the same advantage. The step that broke the build and the step that fixed it get the same mark.
- **Step-level groups:** GiGPO (2025) groups the actions taken from the same state across rollouts and compares them, giving step-level advantages without extra rollouts (about 12 points on ALFWorld, 9 on WebShop over GRPO).
- **Agent process reward models:** a step critic trained from Monte Carlo rollouts (“promise plus progress”), with the same risks as in Lecture 13: noisy labels, reward hacking.
- **The analogy:** one grade for a group project. A fairer grade compares students who faced the same decision and chose differently.
- **In production** most systems still use outcome rewards from verifiable environments plus rubric judges; step-level credit is research.

# Multi-Step Reasoning as Search (MCTS+LLMs)

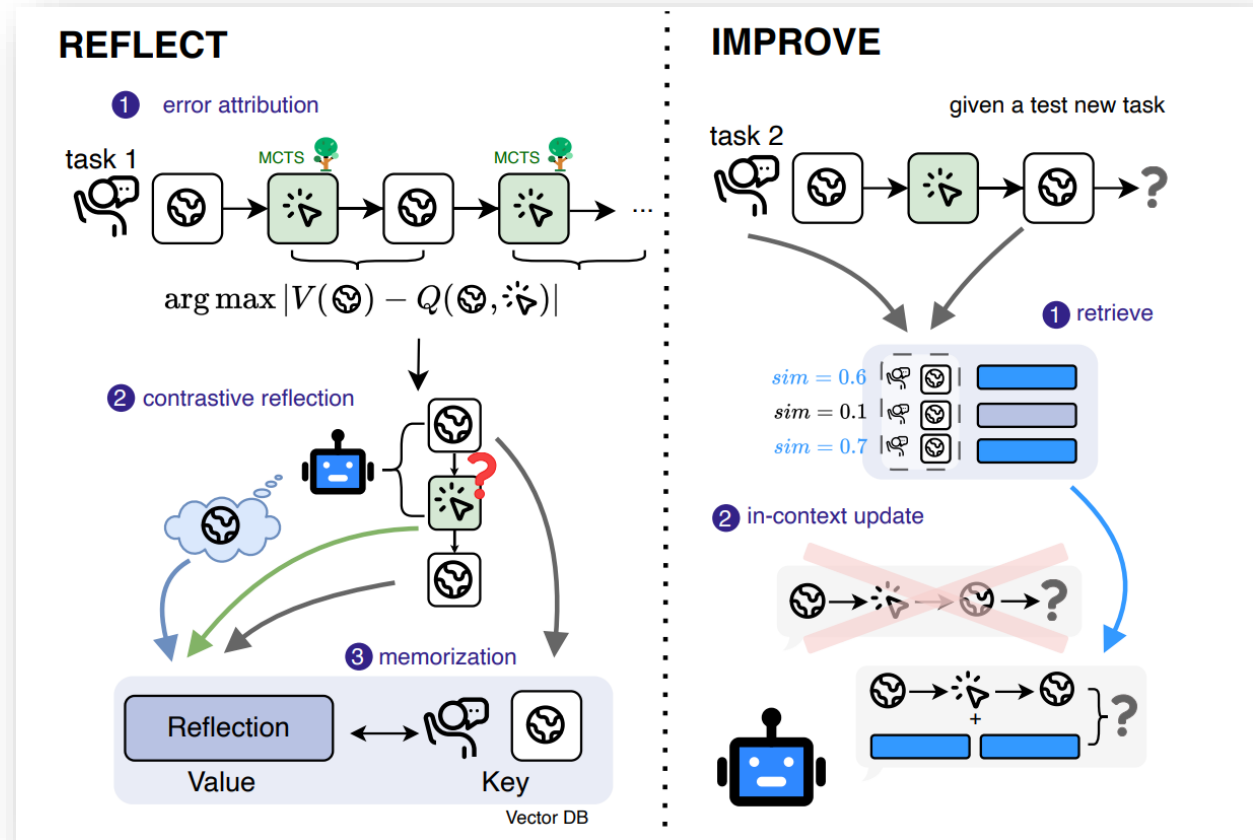
For **complex reasoning**, one can integrate **Monte Carlo Tree Search (MCTS)**:

- **Nodes** = partial reasoning trajectories.
- **Edges** = possible next steps generated by the LLM.
- **Rollouts** = LLM simulates completing the trajectory.
- **Value function** = estimated reward of a reasoning path.

The agent then picks actions with both **policy guidance (LLM)** and **value-based search**.

# R-MCTS: Reflective Monte Carlo Tree Search

- MCTS – a classic search algorithm in AI and gaming research
- Strong roots in Maastricht and at DACS



# Where MCTS works, and where it cannot

- **MCTS needs three things:** an exact state you can reset, a cheap simulator, and a trustworthy value or verifier.
- **Formal mathematics has all three:** AlphaProof (DeepMind) searches Lean proofs with an AlphaZero-style agent and reached silver at the IMO 2024. rStar-Math (2025): MCTS with code-verified steps as a data generator; a 7B model reaches the level of o1-preview.
- **Web and software agents in research:** LATS (MCTS with the model as policy, value and reflector); Agent Q (MCTS plus DPO on a shopping task, 18.6 to 81.7 percent); tree search on WebArena (about 28 percent relative gain); R-MCTS (previous slide).
- **Why you cannot MCTS an email:** no undo, no simulator, no value function. And a model trained with RL closes most of the gap at the cost of one sample (Lecture 13: DeepSeek dropped MCTS for text).
- **2026:** frontier agents run linear interleaved loops with retries and sub-agents; search survives as training-data synthesis and in verifiable domains.



# Capability is not reliability

- **tau-bench (Sierra, 2024):** an agent, a simulated user and a policy document;  $\text{pass}^k$  is the chance of succeeding in all  $k$  runs of the same task. Retail: about 86 percent  $\text{pass}^1$  in 2025, falling fast as  $k$  grows.
- **SWE-bench Verified:** from a few percent in 2023 to about 80 percent in late 2025. A 2026 audit finds the top ten systems solve the same instances and fail the same 51; the benchmark now measures reliability, cost and contamination rather than capability.
- **Every agent benchmark of 2023 and 2024** (WebArena, GAIA, BrowseComp) moved from under 20 percent to human level in two to three years, mostly through better models, not better harnesses.
- **The question for deployment:** would you put an agent in front of customers that succeeds on a given task every other time? Reliability, cost and verification are the open problems, not capability.



# Learning & Adaptation

## Online Adaptation:

- Unlike fine-tuning or RLHF, which are **offline** (done before deployment), **online adaptation** means the system learns **during deployment** from its interactions.
- This is **still very hard** in practice because:
  - LLMs are static at inference time (no built-in memory of past mistakes or user feedback).
  - Adaptation risks "catastrophic forgetting" or divergence.
- **Emerging research directions:**
  - **Process-based rewards:** Rather than rewarding only end results, systems are evaluated on the *reasoning process* they follow.
  - **Self-reflection:** Some agents are equipped with mechanisms to critique their own output, revise, or verify before acting (e.g., Reflexion, DEPS).

# 7. Verification and Guardrails: Fact-checking Modules

- **Problem:** LLMs often produce fluent but factually wrong answers (hallucinations).
- **Solution:** Integrate **retrieval-based verification**.
  - After the LLM generates an answer, the system checks it against authoritative sources (databases, APIs, search engines).
  - Example: If an LLM says “*Paris has 15 million residents*”, the fact-checker queries a population database and flags the mismatch.
- **Methods:**
  - Retrieval-Augmented Verification (RAV) — compare generated claims with retrieved passages.
  - Claim decomposition — break answer into atomic statements and check each.
- **Limitations:** Fact-checking works best for verifiable knowledge (dates, numbers, facts), but less well for reasoning or subjective judgment.

# Verification and Guardrails

## Safety Filters (Ethical and Legal Constraints)

- **Problem:** Agents might produce content that is offensive, illegal, or violates compliance requirements.
- **Solution:** Add **policy filters** that check outputs against rules.
  - **Ethical guardrails:** block hate speech, harassment, misinformation.
  - **Legal guardrails:** compliance with GDPR, HIPAA, copyright.
  - **Domain-specific guardrails:** e.g., financial advice agents restricted from suggesting fraudulent activities.
- **Methods:**
  - Classifiers trained to detect unsafe or disallowed content.
  - Rule-based filters (e.g., regex for credit card leaks).
  - Risk scoring models to flag high-risk outputs for human review.



# Verification and Guardrails

## External Critics or Evaluators

- **Problem:** LLMs can be overconfident and wrong.
- **Solution:** Add **secondary evaluators** that critique or validate outputs.
  - **Self-consistency checks:** Run the same question multiple times and compare answers; discard inconsistent outputs.
  - **Debate agents:** Two or more models argue for and against a solution; a judge agent (or human) picks the most robust reasoning.
  - **Reflexion frameworks:** LLM reviews its own past mistakes, revises its answers, and learns from errors.
- **Benefit:** Adds a second layer of reasoning beyond the first answer, reducing blind trust in a single output.

# Verification and Guardrails

## Why These Matter in Agentic Systems:

- **Error propagation risk:** In a multi-step agent loop, one wrong step can cascade into a chain of failures (e.g., mis-fact → wrong plan → useless final answer).
- **High-stakes contexts:** In law, medicine, finance, or compliance, unchecked outputs can cause real harm.
- **Trustworthiness:** Users (and regulators) won't trust AI systems unless they are backed by auditable, verifiable, and explainable guardrails.

# There is nothing Magic

- LLM-agents are not magic — they fit into the **RL agent paradigm** with states, actions, policies, and rewards.
- The **difference**: instead of a small discrete action space, the LLM acts in a **combinatorial natural-language space** (millions of possible sequences).
- Deep learning enables tractable sampling from this huge policy space.
- The addition of **memory, reasoning loops, and tool calls** makes LLMs genuine **agents**, not just sequence predictors.

**The New York Times**

There's a word that Sam Altman likes to use when talking about artificial intelligence: magic. Last year, he called a version of ChatGPT "magic intelligence in the sky." In February, he referred to "magic unified intelligence." He later posted that a recent update has "a magic to it i haven't felt before."

<https://www.nytimes.com/2025/09/25/opinion/artificial-intelligence-magical-thinking.html>

# Additional References

| Title                                                                                        | Authors / Venue                     | What it covers / why it's useful                                                                                                             |
|----------------------------------------------------------------------------------------------|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| The Rise and Potential of Large Language Model Based Agents                                  | Z Xi et al. (2023)                  | A comprehensive survey tracing the notion of agents and proposing a general framework (brain, perception, action). ( <a href="#">arXiv</a> ) |
| Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects | Cheng et al. (2024, arXiv)          | Defines “LLM agents,” surveys single- & multi-agent systems, tool use, planning, etc. ( <a href="#">arXiv</a> )                              |
| A Survey on Large Language Model based Human-Agent Systems                                   | Zou et al. (2025, arXiv)            | Focuses on human-in-the-loop / human-agent systems, safety, feedback, orchestration. ( <a href="#">arXiv</a> )                               |
| Computational Experiments Meet Large Language Model Based Agents: A Survey and Perspective   | Ma et al. (2024)                    | Connects LLM-agents with agent-based modeling and simulation, especially for social environments. ( <a href="#">arXiv</a> )                  |
| TrustAgent: Towards Safe and Trustworthy LLM-based Agents                                    | Hua et al. (2024)                   | Addresses safety, reliability, constitutional constraints for LLM agents — important for real-world deployment. ( <a href="#">arXiv</a> )    |
| Orchestrated Problem Solving with Multi-Agent LLMs                                           | (2024, arXiv)                       | Describes an orchestrator LLM dividing tasks among specialized agents and coordinating them. ( <a href="#">arXiv</a> )                       |
| An LLM Agent for Comprehensive Academic Paper Search (PaSa)                                  | He et al. (2025, ACL Long)          | Concrete deployed system with two LLM agents (Crawler + Selector) handling search and paper curation. ( <a href="#">ACL Anthology</a> )      |
| Chain-of-Agents: LLM Collaboration for Long-Context Tasks                                    | Google Research blog / paper (2025) | Introduces agent collaboration to handle long contexts by chaining agents together. ( <a href="#">Google Research</a> )                      |

# References: reasoning and RL in agents

- Yao et al. 2022, ReAct: Synergizing Reasoning and Acting in Language Models, arXiv:2210.03629
- Shinn et al. 2023, Reflexion: Language Agents with Verbal Reinforcement Learning, arXiv:2303.11366
- Huang et al. 2023, Large Language Models Cannot Self-Correct Reasoning Yet, arXiv:2310.01798
- Zhou et al. 2024, Language Agent Tree Search (LATS), arXiv:2310.04406; Putta et al. 2024, Agent Q, arXiv:2408.07199
- Koh et al. 2024, Tree Search for Language Model Agents, arXiv:2407.01476; Yu et al. 2024, R-MCTS, arXiv:2410.02052
- Anthropic, December 2024, Building effective agents; September 2025, Effective context engineering for AI agents
- Yao et al. 2024, tau-bench, arXiv:2406.12045; Jimenez et al. 2023, SWE-bench, arXiv:2310.06770
- Guan et al. 2025, rStar-Math, arXiv:2501.04519; DeepMind 2025, AlphaProof, Nature
- Wei et al. 2025, SWE-RL, arXiv:2502.18449; Jin et al. 2025, Search-R1, arXiv:2503.09516; Feng et al. 2025, ReTool, arXiv:2504.11536
- OpenAI, April 2025, Introducing o3 and o4-mini; Moonshot AI 2025, Kimi K2, arXiv:2507.20534; Qwen team 2025, Qwen3-Coder
- Feng et al. 2025, GiGPO, arXiv:2505.10978; Choudhury 2025, Process Reward Models for LLM Agents, arXiv:2502.10325
- Zhang et al. 2025, The Landscape of Agentic Reinforcement Learning for LLMs, arXiv:2509.02547
- Brilliantov, Hernandez-Cano, Abbe 2026, How Much of a Harness Does a Strong Agent Need?, arXiv:2609.40303



# Questions?



Maastricht University

# Exercise 1: From "Brains in a Jar" to Formal Agent Policies

Review the sections comparing 1990s agents to modern LLM agents and the formal definition of agents using Partially Observable Markov Decision Processes (POMDPs).

- The lecture describes a stand-alone LLM as a "brain in a jar". What specific architectural components must be wrapped around this "brain" to give it a functional "body" and transform it into an autonomous agent?
- When formalizing an LLM agent using reinforcement learning terminology, how are the traditional concepts of *State* ( $s_t$ ), *Action* ( $a_t$ ), and *Policy* ( $\pi_\theta$ ) specifically reinterpreted for language models?
- Contrast the goal management strategies of 1990s BDI (Belief-Desire-Intention) agents with modern LLM agents. Despite the flexibility of natural language goal setting in modern agents, what fundamental goal management challenges (such as drift and conflict resolution) remain difficult to solve?

## Exercise 2: Advanced Reasoning Loops and Self-Correction

Review the slides on ReAct (Reasoning + Acting), Reflexion, AutoGPT, interleaved thinking and agentic RL.

- The ReAct framework interleaves reasoning and tool use into a continuous loop. Detail the specific, repeating steps of the ReAct loop (Thought, Action, Observation). Why does combining reasoning and acting in this way prevent the agent from getting stuck compared to "reason-only" or "act-only" prompting?
- Explain the concept of *Reflexion* as a mechanism for learning and adaptation. How does generating a "verbal reflection" after a failed task attempt serve as a form of meta-cognition that allows the agent to correct its future behavior?
- How does an autonomous framework like AutoGPT differ from traditional Chain-of-Thought (CoT) prompting? Specifically, explain how AutoGPT utilizes a goal-setting interface, autonomous planning, and self-reflection to operate without constant user input.

## Exercise 3: Memory Systems, Verification, and Guardrails

Review the sections covering the components of an LLM agent, specifically Memory and Verification & Guardrails.

- Modern LLM agents require multiple types of memory to overcome the limitations of a static context window. Define and provide an example for each of the following memory types: Short-term memory, Long-term memory, Episodic memory, and Semantic memory.
- To combat hallucinations and logical errors, agentic systems employ a Chain of Verification (CoVe). Outline a hypothetical multi-stage CoVe pipeline that an agent might use to verify an answer, incorporating document retrieval, information extraction, and output validation.
- In high-stakes environments, agents must include strict safety filters. What are the differences between ethical, legal, and domain-specific guardrails? Explain how external critics (such as "debate agents" or self-consistency checks) can be implemented to enforce these guardrails before an output is surfaced to the user.

# Discussion



# On Goals: Can a Next-Token Predictor Keep a Promise?

The agents of the 1990s managed goals with explicit rules and symbolic logic. They were precise, verifiable, and so brittle that they could not survive contact with real language. LLM agents handle real language effortlessly and manage goals by generating the next most likely sentence.

Have we traded ambiguity paralysis for confident hallucination? When two goals conflict inside an LLM agent, who decides, and would anyone be able to tell afterwards how the decision was made?

Can a probabilistic text generator ever give the guarantees the old systems gave (this goal always wins; that action never happens), or is “guarantee” a word that only applies to systems too rigid to be useful?

# On Autonomy and Memory: Is an Agent That Forgets Still the Same Agent?

An LLM agent's state is its context window. Clear the window and the agent has no past, no plan, no self; what remains is a vector store it may or may not consult. The next session starts a new agent with the same weights.

Is an agent that forgets everything between episodes one agent, or a sequence of short-lived agents sharing a brain? What makes an agent the same agent over time: its weights, its memory, its goals, or someone's decision to treat it as one?

Autonomy is usually understood as acting from one's own goals over time. If the goals live in a prompt that someone else writes and a memory that someone else can edit, in what sense is anything here autonomous, and who is responsible for what it does?

# On Reflexion: Self-Correction, or a Critic Persona That Happens to Work?

Reflexion lets an agent read its own failed attempt, write a paragraph about what went wrong, and try again with that paragraph in the prompt. It works. The framework calls it meta-cognition.

The weights that failed are the weights that now diagnose the failure. Why should the same model be better at judging its work than at doing it? And if it is, what does that say about where the failure was: in the model's ability, or in the path it happened to take?

Is self-correction here a cognitive process, or a prompt that shifts the model into a critic persona which happens to unlock a better path? Would the distinction matter if the results were identical, and to whom?